1. Criar uma Hierarquia de Produtos e Filtrar por Preço

Crie uma **interface** Vendavel com o método double getPreco().
Crie uma **classe abstrata** ItemLoja com os atributos id e nome, implementando Vendavel.
Crie duas classes concretas Eletronico e Roupas, herdando ItemLoja.

✅ **Objetivo:**

- Criar uma **lista intermediária** contendo apenas os nomes dos produtos com preço acima de R$ 100,00 usando **filter()** e **map()**.
- Converter a lista de produtos para uma **lista de descrições** usando .map(ItemLoja::getNome).

---

2. Ordenação de Objetos (ASC e DESC) e Transformação de Lista

Crie uma classe Aluno com os atributos String nome, double nota e int idade.

✅ **Objetivo:**

- Criar uma **lista intermediária** com alunos aprovados (**nota >= 7**).
- Ordená-los **por nota (decrescente)** e **idade (crescente)** usando .sorted(Comparator.comparing(Aluno::getNota).reversed()).
- Transformar a lista de alunos em uma **lista de nomes** usando .map(Aluno::getNome).

---

3. Agrupamento de Funcionários por Cargo e Mapeamento para Médias

Crie uma **classe abstrata** Pessoa com nome e idade.
Crie uma **classe concreta** Funcionario, herdando Pessoa e adicionando String cargo e double salario.

✅ **Objetivo:**

- **Agrupar funcionários por cargo** em um Map<String, List<Funcionario>> usando **Collectors.groupingBy()**.
- Criar uma **lista intermediária** contendo apenas os nomes dos funcionários usando **map()**.
- Criar um **Map<String, Double>** contendo a média salarial por cargo usando **Collectors.averagingDouble(Funcionario::getSalario)**.

---

4. Mapear uma Lista de Objetos para um DTO

Crie uma classe Pedido com os atributos int id, String cliente, double valorTotal. Crie um DTO (PedidoDTO) que contenha apenas id e valorTotal.

✅ **Objetivo:**

- Criar uma **lista de PedidoDTO a partir de uma lista de Pedido** usando .map(p -> new PedidoDTO(p.getId(), p.getValorTotal())).

---

5. Redução e Soma de Valores Totais com Mapeamento

Crie uma classe Produto com os atributos String nome e double preco.

✅ **Objetivo:**

- Criar uma **lista intermediária** contendo apenas os preços dos produtos usando **map()**.
- Somar os preços com **Collectors.summingDouble()**.
- Calcular a média de preços com **Collectors.averagingDouble()**.

---

6. Filtragem, Contagem e Transformação de Objetos

Crie uma classe Cliente com String nome e int idade.

✅ **Objetivo:**

- Criar uma **lista intermediária** com clientes **maiores de 18 anos**.
- Contar **quantos clientes possuem mais de 30 anos** usando .count().
- Criar uma **lista apenas com os nomes dos clientes** usando .map(Cliente::getNome).

---

7. Encontrar o Maior e o Menor Valor de uma Lista com Mapeamento

Crie uma lista de Produto e utilize **Collectors.maxBy()** e **Collectors.minBy()** para encontrar:

✅ **Objetivo:**

- O produto **mais caro** e **mais barato**.

- Criar uma **lista intermediária** contendo apenas os nomes dos produtos usando .map(Produto::getNome).

---

8. Concatenar Strings de uma Lista e Criar uma Representação Textual

Crie uma **interface Descritivel** com um método String getDescricao().
Crie uma **classe Pessoa** implementando Descritivel.

✅ **Objetivo:**

- Criar uma **lista intermediária** contendo apenas descrições das pessoas usando .map(Pessoa::getDescricao).
- Concatenar todas as descrições em uma única string separada por vírgulas usando **Collectors.joining(", ")**.

9. Conversão de Lista de Objetos para um Map Baseado em Identificadores

Crie uma classe Produto com os atributos int id, String nome, double preco.

✅ **Objetivo:**

- Converter uma lista de Produto em um **Map<Integer, Produto>**, onde a chave é o id do produto.
- Utilize .collect(Collectors.toMap(Produto::getId, p -> p)).

---

10. Filtragem de Dados e Cálculo Estatístico

Crie uma classe Vendedor com os atributos String nome, double totalVendas.

✅ **Objetivo:**

- Criar uma **lista intermediária** contendo apenas os vendedores que venderam **mais de R$ 10.000,00**.
- Encontrar o **maior e o menor total de vendas** usando Collectors.maxBy() e Collectors.minBy().
- Criar uma **lista contendo apenas os nomes dos vendedores** usando .map(Vendedor::getNome).

---

11. Criar e Agrupar Objetos por Status

Crie uma classe Pedido com os atributos int id, String cliente, String status ("Pendente", "Pago", "Cancelado").

✅ **Objetivo:**

- Criar uma **lista intermediária** contendo apenas os pedidos com status "Pago".
- Agrupar os pedidos por status em um **Map<String, List<Pedido>>** usando Collectors.groupingBy(Pedido::getStatus).

---

12. Criar e Transformar Listas de Funcionários para Salários Ajustados

Crie uma classe Funcionario com os atributos String nome, double salario, String departamento.

✅ **Objetivo:**

- Criar uma **lista intermediária** contendo apenas funcionários do departamento "TI".
- Aplicar um **aumento de 10% nos salários** usando .map(f -> new Funcionario(f.getNome(), f.getSalario() * 1.1, f.getDepartamento())).
- Criar um **Map<String, Double>** contendo o total dos salários por departamento usando .collect(Collectors.groupingBy(Funcionario::getDepartamento, Collectors.summingDouble(Funcionario::getSalario))).

---

13. Criar, Ordenar e Transformar Objetos de Uma Lista

Crie uma classe Livro com os atributos String titulo, String autor, double preco.

✅ **Objetivo:**

- Ordenar a lista de livros por **preço em ordem crescente** e depois por **autor em ordem alfabética**.
- Criar uma **lista intermediária** contendo apenas os títulos dos livros usando .map(Livro::getTitulo).
- Criar uma **única string contendo os títulos separados por " | "** usando .collect(Collectors.joining(" | ")).